02 - System Requirements Specification

Visor DOM-to-Agent Context Compiler
DOC-02 | 0.1 requirements pack | Generated 2026-06-14

This SRS defines functional and non-functional requirements, system interfaces, constraints, and acceptance signals for the MVP.

Field	Value
Owner	Rohan PX / Visor project
Audience	AI coding agent, reviewer, future contributors
Status	Draft baseline for MVP build
Decision rule	MVP is local-first, minimal-permission, Chrome Manifest V3, no backend unless explicitly enabled.

Document control

Item	Value
Project	Visor DOM-to-Agent Context Compiler
Document code	DOC-02
Generated on	2026-06-14
Primary goal	Turn raw webpage DOM into safe, structured, agent-ready context.
MVP boundary	Chrome extension, local-only compiler, JSON/Markdown export, privacy redaction, tests.

1. System context

The system consists of a browser extension shell, a content-script based DOM extractor, a service-worker coordination layer, a compiler pipeline, a privacy redaction engine, a preview/export UI, and local settings storage. The MVP has no backend dependency.

2. Functional requirements

ID	Requirement	Priority	Acceptance signal
FR-001	Detect the current active tab and request extraction only after user action.	P0	Compile button starts extraction for active tab.
FR-002	Extract URL, title, timestamp, language hints, and metadata.	P0	PageSnapshot.source is populated.
FR-003	Extract visible text blocks with hierarchy and selector hints.	P0	Fixture pages return ordered visible blocks.
FR-004	Extract headings h1-h6 with level and text.	P0	Heading tree is present.
FR-005	Extract links with text, href, rel/target when available.	P0	Links list includes normalized href.
FR-006	Extract buttons and actionable controls with labels and selector hints.	P0	ActionableElements include buttons and purpose guess.
FR-007	Extract forms, inputs, labels, placeholders, and field types.	P0	Form model exists for form-heavy fixture.
FR-008	Extract tables as rows, cells, headers, caption, and surrounding context.	P0	Table-heavy fixture outputs structured rows.
FR-009	Extract code blocks and preserve formatting enough for agent context.	P1	Docs fixture includes code blocks.
FR-010	Filter scripts, styles, hidden nodes, empty nodes, and repeated boilerplate.	P0	Debug output marks removed blocks.
FR-011	Deduplicate repeated text across nav, sidebar, footer, and repeated cards.	P0	Repeated tokens are counted as removed noise.
FR-012	Classify page type into article, docs, dashboard, product, social, form, table, app, or unknown.	P1	PageClassification is populated with confidence.
FR-013	Estimate tokens for extracted and compiled content.	P0	TokenProfile appears in every output.
FR-014	Generate schema-valid AgentContext JSON.	P0	Validator passes all fixture outputs.
FR-015	Generate Markdown and prompt-block exports.	P0	Clipboard exports match selected format.
FR-016	Apply privacy redaction based on user-selected level.	P0	Strict mode masks fixture secrets.
FR-017	Persist user settings locally.	P0	Settings survive browser restart.
FR-018	Support debug mode with kept/removed blocks and compiler notes.	P1	Debug inspector shows reasons.

3. Non-functional requirements

ID	Requirement	Priority	Acceptance signal
NFR-001	Normal article compilation should complete in under 2 seconds on a modern laptop.	P0	Performance test passes median target.
NFR-002	Long documentation pages should compile without freezing the browser.	P0	Chunked processing and node limits are used.
NFR-003	MVP must make zero external network calls during compile.	P0	Network inspection test confirms no calls.
NFR-004	Extension permissions must be minimal and documented.	P0	Manifest review has no broad host permission unless justified.
NFR-005	Output schema must be deterministic for the same DOM snapshot and settings.	P0	Repeat compile diff is stable except timestamp IDs.
NFR-006	Errors must degrade gracefully and show user-readable messages.	P0	Restricted page test displays actionable error.
NFR-007	All modules must be type-safe and testable.	P0	Typecheck and unit tests pass.
NFR-008	No unsafe DOM rendering of extracted content in extension UI.	P0	UI uses textContent/escaped rendering, not raw innerHTML.

4. Component interfaces

```
type CompileRequest = {
  mode: 'compact' | 'detailed' | 'agent_action' | 'rag' | 'debug';
  privacyLevel: 'low' | 'medium' | 'strict';
  tokenBudget: number;
  siteProfile?: SiteProfile;
};

type CompileResponse = {
  ok: true;
  snapshot: PageSnapshot;
  context: AgentContext;
  exports: { json: string; markdown: string; promptBlock: string };
} | {
  ok: false;
  errorCode: string;
  userMessage: string;
  debug?: unknown;
};
```

5. Constraints and assumptions

- The extension cannot run on browser internal pages such as chrome:// URLs.
- Some cross-origin iframes may be inaccessible; they should be reported, not guessed.
- Shadow DOM support may be partial in MVP, but the extractor should record when shadow roots are skipped or traversed.
- The system does not guarantee perfect main-content detection; debug tools and site profiles compensate.
- External AI summarization is not used in MVP; summaries are extractive or rule-based placeholders unless later enabled explicitly.

Source and reference notes

These sources are used to ground platform/security assumptions. The implementation must still verify current platform behavior during development.

Reference	Why it matters
Chrome Extensions - Manifest V3 and service workers: https://developer.chrome.com/docs/extensions/	Grounding for MV3, background service worker model, permissions, and packaging.
MDN WebExtensions - Content scripts: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_scripts	Grounding for content script behavior, host permissions, and page DOM access.
MDN WebExtensions - storage API: https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/API/storage	Grounding for extension storage concepts and permissions.
OWASP Browser Extension Vulnerabilities Cheat Sheet: https://cheatsheetseries.owasp.org/cheatsheets/Browser_Extension_Vulnerabilities_Cheat_Sheet.html	Grounding for permissions overreach, data leakage, XSS, and insecure communication mitigations.
OWASP LLM Top 10 - LLM01 Prompt Injection: https://genai.owasp.org/llmrisk/llm01-prompt-injection/	Grounding for treating webpage content as untrusted data in agent pipelines.